

Module 4 — Subagents

Lesson 4.5

Worktree isolation

Spawn a subagent in a temporary git worktree. It edits an isolated copy of the repo; you compare before merging anything.

Why it matters

Parallel agents that all write to the same working tree corrupt each other's edits. Even sequentially, you often want to attempt a risky change in a sandbox before letting it touch your real branch. Worktree isolation gives you that — cheaply, with full git history.

What a git worktree is

A `git worktree` is an additional checkout of the same repo, in a different directory, on a different branch. They share `.git/` but have independent working files. Switching branches in one doesn't affect the other.

For Claude Code, the relevant fact: the harness can spawn a subagent whose working directory is a freshly created worktree on a fresh branch.

Using it

When invoking the Agent tool, set `isolation: "worktree"`:

```
Agent(  
  description: "try migration approach B",  
  subagent_type: "general-purpose",  
  isolation: "worktree",  
  prompt: "Implement the user-table migration using approach B (dual-write + cutover). Run the  
    migration tests. Report what worked and what didn't."  
)
```

Behind the scenes:

- 1 The harness creates a worktree (e.g., `../<repo>-agent-abc123`) on a new branch.
- 2 The agent works there. Its `git status`, `git diff`, `git log` all see the worktree state.
- 3 On agent completion, the harness reports the worktree path and branch so you can inspect.
- 4 If the agent made no changes, the harness cleans up the worktree automatically.

When to use it

Situation	Worktree isolation?
Read-only research	No (overhead without benefit)
One-shot small edit	No (just use a normal agent)
Try a risky approach you might discard	Yes
Compare two implementations of the same change	Yes (one worktree each)
Parallel agents that both write to overlapping code	Yes (one worktree each)
Long-running speculative refactor	Yes (don't pollute your main tree)

The compare-implementations pattern

Agent 1 (worktree A): "Implement the cache as a Redis store."

Agent 2 (worktree B): "Implement the cache as an in-memory LRU."

Then YOU diff both worktrees against main, run the benchmarks in each, and pick – without ever having those changes in your main tree.

The closest thing to "try two designs in parallel without committing to one" we have in tooling. Underused.

Inspecting agent worktrees

After a worktree agent returns, the result includes the worktree path and branch. You can:

- `cd` into the worktree directly
- `git log <branch>` to see what was committed (if anything)
- `git diff main..<branch>` to see the net change
- Cherry-pick interesting commits back to your main work

```
# from inside your main worktree
git diff main..agent-abc123-branch -- src/
```

```
# or
git worktree list
cd ../yourrepo-agent-abc123
```

Cleanup

If the agent made changes you want to keep, merge or cherry-pick. Then remove the worktree:

```
git worktree remove ../yourrepo-agent-abc123
# or with -f if there are uncommitted changes you've accepted as lost
```

If the agent made no changes, the harness already cleaned up. No action needed.

What goes wrong without isolation

- Two agents editing `package.json` simultaneously: the second overwrites the first.
- An agent's experimental refactor leaves your working tree dirty when you try to commit unrelated work.
- An agent fails partway and leaves the repo in a broken intermediate state.

All avoided by isolating.

Try it

- 1 Pick a refactor you've been putting off because it might not work.
- 2 Spawn a general-purpose agent with `isolation: worktree` and brief it to attempt the refactor.
- 3 When it returns, `cd` into the worktree, run tests, decide whether to keep.
- 4 Either merge to your branch or `git worktree remove` and forget it ever happened.

Gotchas

- **Worktrees consume disk** (one full checkout per worktree). Not free on huge repos.
- **Shared `.git/` means shared hooks.** A `pre-commit` hook fires in the worktree too. Usually fine, sometimes surprising.
- **Branches created in worktrees stick around** until you delete them, even after `worktree remove`. Periodic `git branch -D <agent-branches>` is sensible cleanup.
- **Cross-platform paths** — on Windows, worktree paths can include spaces; quote everything.
- **Submodules** in worktrees behave oddly in some git versions; verify if your repo uses them.

Further reading

- `git worktree --help` — the underlying primitive
- 4.4 Parallel orchestration — when to combine with parallelism

Next

→ 5.1 MCP overview